Treinamento e Avaliação de um Modelo de Linguagem Estilo GPT-2 sobre a Obra Completa de Machado de Assis

Roberto [SOBRENOME] Matrícula: [MATRÍCULA]
Departamento de Ciência da Computação, Universidade de Brasília
Redes Neurais e Aprendizado Profundo, 2026/2 — Projeto Individual 1 — Prof. Díbio
Código e *notebook*: https://github.com/RobertorNeto/MachadoGPT

***Resumo*—Este trabalho descreve a construção de um corpus com a obra original de Machado de Assis e o treinamento e a avaliação de modelos de linguagem autorregressivos baseados na arquitetura GPT-2, implementados com o nanoGPT. O corpus foi obtido a partir da *Obra Completa* distribuída pelo NLTK, limpo por um pipeline próprio e dividido por obra inteira, com estratificação por gênero, em conjuntos de treino, validação e teste. Três experimentos foram comparados sob o mesmo orçamento de treino: um modelo em nível de caractere treinado do zero (E1), um modelo com o tokenizador BPE do GPT-2 treinado do zero (E3) e o *fine-tuning* do GPT-2 pré-treinado de 124M de parâmetros (E6). Como as perdas não são comparáveis entre tokenizadores, os modelos foram comparados em bits por caractere (BPC). Os resultados de validação foram 1,737 BPC (E1), 1,653 BPC (E3) e 1,586 BPC (E6), mostrando que tanto a tokenização por subpalavras quanto o pré-treinamento melhoram a modelagem do texto, ainda que o GPT-2 original, treinado majoritariamente em inglês, apresente 2,975 BPC antes do ajuste. As amostras geradas reproduzem traços estilísticos do autor, como diálogos com travessão e colocação pronominal de época, mas sem coerência narrativa de longo alcance. Por fim, discute-se como o sistema poderia ser estendido para responder a perguntas no universo machadiano por meio de geração aumentada por recuperação e de ajuste supervisionado.**

***Index Terms*—modelos de linguagem, Transformer, GPT-2, nanoGPT, Machado de Assis, processamento de linguagem natural**

I. Introdução

Modelos de linguagem autorregressivos baseados na arquitetura Transformer [1] tornaram-se a base dos sistemas modernos de geração de texto. A família GPT [2], [3] demonstrou que um único objetivo de treino — prever o próximo token de uma sequência — é suficiente para que um modelo aprenda regularidades ortográficas, sintáticas e estilísticas de um corpus, e que o conhecimento adquirido em grandes coleções de texto pode ser transferido para domínios específicos.

Este projeto aplica essa abordagem à obra de Machado de Assis (1839–1908), cujos textos estão em domínio público. O problema abordado tem duas partes. A primeira é de engenharia de dados: obter, limpar e organizar um corpus com o conteúdo original dos livros do autor, adequado ao treinamento de um modelo de linguagem. A segunda é de modelagem: treinar modelos similares ao GPT-2 sobre esse corpus, usando o nanoGPT [4] como código base, e comparar escolhas de projeto que afetam a qualidade do modelo resultante.

Três perguntas orientam a comparação experimental:

1) Um GPT pequeno treinado do zero, em nível de caractere, consegue capturar o estilo do autor?
2) Substituir caracteres por subpalavras (tokenizador BPE do GPT-2), mantendo o mesmo modelo, melhora a modelagem?
3) Partir dos pesos pré-treinados do GPT-2, majoritariamente em inglês, é melhor do que treinar do zero em português?

As contribuições são: (i) um pipeline reprodutível de construção do corpus, com tratamento de codificação, cabeçalhos editoriais e sumários; (ii) uma divisão treino/validação/teste por obra inteira e estratificada por gênero, que evita vazamento entre conjuntos; (iii) a comparação de três configurações sob o mesmo orçamento de treino, com uma métrica comparável entre tokenizadores; e (iv) uma discussão sobre a extensão do sistema para responder a perguntas sobre o universo machadiano.

II. Fundamentação

A. Transformer e atenção

O Transformer [1] substitui a recorrência por mecanismos de atenção. Em cada camada, cada posição da sequência calcula uma média ponderada das representações das demais posições, com pesos dados por

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad (1)$$

em que Q, K e V são projeções lineares das entradas. A atenção *multi-cabeça* executa esse cálculo em paralelo em subespaços distintos, permitindo que cada cabeça capture um tipo diferente de relação (concordância, pontuação, referência entre falas de um diálogo). Cada bloco combina atenção, uma rede *feed-forward* posicional, conexões residuais e normalização de camada.

B. Modelos GPT

O GPT-2 [3] é um Transformer somente decodificador: uma máscara causal impede que a posição t atenda a posições futuras, e o modelo é treinado para maximizar $\sum_t \log p(x_t \mid x_{<t})$. O treino é auto-supervisionado, pois o próprio texto fornece o alvo de cada posição. O GPT-2 foi lançado em quatro tamanhos (124M a 1,5B de parâmetros), todos com tokenizador *Byte Pair Encoding* (BPE) [5] de 50 257 tokens e

contexto de 1024 tokens. O GPT-3 [2] escalou a mesma receita para 175B de parâmetros e evidenciou que o conhecimento do pré-treinamento pode ser reaproveitado com pouco ou nenhum ajuste, motivação direta do experimento de *fine-tuning* deste trabalho.

C. nanoGPT

O nanoGPT [4] é uma reimplementação compacta do GPT-2 em PyTorch, com cerca de 300 linhas para o modelo (`model.py`) e um laço de treino (`train.py`) configurável por arquivos Python. A arquitetura do `model.py` é idêntica à do GPT-2, o que permite tanto treinar modelos do zero em tamanhos arbitrários quanto carregar os pesos originais da OpenAI e continuar o treino.

III. Construção do Corpus

A. Fonte

O texto bruto foi obtido do pacote `machado` do NLTK Data [6], que reproduz a *Obra Completa* publicada pelo Ministério da Educação (`machado.mec.gov.br`), com texto de referência da edição Nova Aguilar de 1994 [7]. O pacote contém 246 arquivos em oito categorias: contos (137), crônica (24), romance (10), tradução (3), crítica (45), poesia (7), teatro (10) e miscelânea (10). Todas as obras são de domínio público.

B. Pipeline de limpeza

A inspeção dos arquivos revelou quatro problemas, tratados por um pipeline próprio:

1) **Codificação.** Os arquivos estão em Windows-1252, não em UTF-8 nem Latin-1. Decodificá-los como Latin-1, prática comum, descarta silenciosamente o travessão (byte `0x97`) e as aspas curvas, o que apagaria justamente o marcador de diálogo característico do autor. A decodificação correta preservou 841 travessões só em *Dom Casmurro*.
2) **Quebras de linha rígidas.** O texto herdou do HTML de origem quebras de linha no meio das frases. Os blocos separados por linha em branco foram reunidos em parágrafos contínuos, com normalização Unicode (NFC) e remoção de caracteres invisíveis.
3) **Cabeçalhos editoriais.** Cada arquivo inicia com a linha "Gênero, Título, Ano", o título e um bloco bibliográfico de formato variável ("Texto-fonte", editora, ano de publicação). Uma heurística localiza, entre os 20 primeiros parágrafos, o último que se assemelha a uma referência bibliográfica e remove tudo até ele.
4) **Sumários.** Coletâneas e livros de poesia trazem um "ÍNDICE" seguido da lista de títulos. A heurística detecta o rótulo (ou uma sequência inicial de linhas curtas) e remove o trecho até a reocorrência do primeiro item, que marca o início do conteúdo. Foram removidos 33 sumários.

As três traduções (Victor Hugo, Charles Dickens e Émile de Girardin) foram excluídas por não serem texto original do autor. Advertências, prefácios e notas escritas pelo próprio Machado foram mantidos. Cada obra foi salva como um arquivo UTF-8 com um parágrafo por linha, acompanhada de metadados (gênero, título, ano, caracteres, palavras). A Tabela I resume o corpus resultante.

Tabela I
Corpus limpo, por gênero (sem traduções).

Gênero	Obras	Caracteres	Palavras
Contos	137	5 267 538	923 040
Crônica	24	3 100 653	539 159
Romance	10	3 056 273	535 111
Crítica	45	940 837	161 485
Poesia	7	652 965	116 619
Teatro	10	362 916	64 074
Miscelânea	10	67 418	11 615
Total	**243**	**13 448 600**	**2 351 103**

Tabela II
Divisão por obra, em caracteres (percentual dentro do gênero).

Gênero	Treino	Validação	Teste
Contos	4 213 760 (80%)	526 565 (10%)	527 213 (10%)
Crônica	2 480 351 (80%)	310 070 (10%)	310 232 (10%)
Romance	2 434 948 (80%)	280 995 (9%)	340 330 (11%)
Crítica	753 183 (80%)	93 906 (10%)	93 748 (10%)
Poesia	542 057 (83%)	70 125 (11%)	40 783 (6%)
Teatro	294 626 (81%)	31 329 (9%)	36 961 (10%)
Miscelânea	53 865 (80%)	6 506 (10%)	7 047 (10%)
Total	**10 772 790**	**1 319 496**	**1 356 314**
Obras	118	42	83

C. Divisão treino/validação/teste

A divisão usual do nanoGPT (os primeiros 90% do texto para treino, o restante para validação) seria inadequada aqui por dois motivos: uma mesma obra ficaria parcialmente em ambos os conjuntos, permitindo que o modelo já conhecesse personagens e enredo do texto de validação (vazamento), e, como as obras estão ordenadas por gênero, a validação seria composta quase só de teatro.

Optou-se por dividir **por obra inteira**, com estratificação por gênero e metas de 80%/10%/10% dos caracteres. Como os tamanhos das obras variam de poucos milhares a centenas de milhares de caracteres, um sorteio aleatório produzia proporções muito desbalanceadas (em um teste, uma única coletânea de crônicas ocupou metade da validação). Foi então usado um algoritmo guloso determinístico: dentro de cada gênero, as obras são ordenadas da maior para a menor e cada uma é atribuída ao conjunto que deixa a distribuição de caracteres mais próxima da meta. A Tabela II mostra o resultado. Os grandes romances (*Dom Casmurro*, *Memórias Póstumas de Brás Cubas*, *Quincas Borba*) ficaram no treino; *Memorial de Aires* ficou na validação e *Ressurreição* e *Casa Velha*, no teste. Uma consequência do algoritmo é que validação e teste concentram obras menores (42 e 83 obras, contra 118 no treino).

Tabela III
CONFIGURAÇÃO DOS EXPERIMENTOS.

	E1	E3	E6
Tokenização	caractere	BPE GPT-2	BPE GPT-2
Pesos iniciais	aleatórios	aleatórios	GPT-2 (124M)
Camadas / cabeças / dim.	6 / 6 / 384	6 / 6 / 384	12 / 12 / 768
Parâmetros	10,7M	29,9M	123,7M
Vocabulário	144	50 304	50 257
Contexto (tokens)	256	256	256
Contexto (caract. aprox.)	256	648	648
Dropout	0,2	0,2	0,1
Taxa de aprendizado	10^{-3}	10^{-3}	10^{-4}
Taxa mínima	10^{-4}	10^{-4}	10^{-5}
Batch × acumulação	64×1	64×1	8×8
Iterações	5000	5000	5000
Passagens pelo treino	7,6	19,3	19,3

IV. METODOLOGIA

A. Tokenização

Foram usadas duas tokenizações. Na primeira, em **nível de caractere**, adaptada do exemplo `shakespeare_char` do nanoGPT, o vocabulário é construído apenas com os caracteres do conjunto de treino (143 símbolos) mais um token de desconhecido, totalizando 144. Essa modificação mantém a separação entre conjuntos; na prática, apenas um caractere da validação não ocorre no treino. O treino tem 10 773 024 tokens.

Na segunda, o texto é codificado com o **BPE do GPT-2** (biblioteca `tiktoken`), o mesmo tokenizador usado no pré-treinamento da OpenAI, o que é obrigatório para o experimento de *fine-tuning*. Seguindo o pré-processamento do GPT-2, cada obra é encerrada com o token especial de fim de texto. Como esse vocabulário foi induzido de texto em inglês, ele fragmenta o português: a frase "Uma noite destas, vindo da cidade" é cortada em `U|ma| no|ite| dest|as|,| vind|o| da| c|id|ade`. A razão observada foi de 2,53 caracteres por token, resultando em 4 248 620 tokens de treino, 524 116 de validação e 535 731 de teste.

B. Experimentos

Os três experimentos foram projetados para que cada um mude uma única variável em relação ao anterior (Tabela III). A diferença entre E1 e E3 isola o efeito da tokenização; a diferença entre E3 e E6 isola o efeito do pré-treinamento. Todos usaram o mesmo orçamento de 5000 iterações com 16 384 tokens por iteração (64 sequências de 256 tokens), em uma GPU NVIDIA T4 do Google Colab, com precisão `float16`.

Os hiperparâmetros de E1 e E3 (6 camadas, 6 cabeças, 384 dimensões, *dropout* de 0,2, taxa de aprendizado de 10^{-3} com aquecimento de 100 iterações e decaimento cosseno até 10^{-4}, AdamW [8] com $\beta_2 = 0{,}99$) foram baseados no exemplo `shakespeare_char` do nanoGPT, um ponto de partida conhecido para GPUs de uma única placa. Em E6, o tamanho do modelo é fixado pelos pesos do GPT-2 (o nanoGPT corta o contexto original de 1024 para 256 tokens); a taxa de aprendizado dez vezes menor evita destruir o conhecimento pré-treinado, e o *batch* de 8 sequências com 8 passos de acumulação de gradiente mantém os 16 384 tokens por iteração dentro da memória da T4. Em E3, o vocabulário de 50 mil tokens acrescenta cerca de 19M de parâmetros de *embedding*, de modo que E3 e E1 diferem também em tamanho e em contexto efetivo, limitação discutida na Seção V-D.

Tabela IV
RESULTADOS NA VALIDAÇÃO. A PERDA DE E1 É POR CARACTERE; AS DEMAIS, POR TOKEN BPE.

Modelo	Perda val.	Perda treino	**BPC**	ms/iter
GPT-2 sem ajuste	5,192	—	2,975	—
E1 (caractere, do zero)	1,204	1,124	1,737	121
E3 (BPE, do zero)	2,885	2,384	1,653	317
E6 (GPT-2 + ajuste)	2,767	2,463	**1,586**	1 145

Uma primeira execução de E6 com 2000 iterações e taxa de aprendizado de 6×10^{-5} resultou em 1,725 BPC, com a perda de validação ainda em queda ao final. Como o orçamento era menor que o de E3, a comparação seria injusta; a execução foi repetida com as 5000 iterações e a taxa de 10^{-4} da Tabela III, e é essa a reportada.

C. Métrica

O nanoGPT reporta a entropia cruzada média por token, estimada a cada 250 (E1, E3) ou 500 (E6) iterações em 200 *batches* de treino e de validação. Essa perda não é comparável entre tokenizadores: um token de E1 é um caractere, enquanto um token de E3/E6 vale 2,53 caracteres em média. Os modelos foram por isso comparados em **bits por caractere** (BPC),

$$\mathrm{BPC} = \frac{L \cdot N_{\mathrm{tok}}}{N_{\mathrm{car}} \cdot \ln 2}, \tag{2}$$

em que L é a perda média por token (em nats), N_{tok} o número de tokens e N_{car} o de caracteres do conjunto avaliado. No caso de E1, $N_{\mathrm{tok}} = N_{\mathrm{car}}$ e a expressão se reduz a $L/\ln 2$. Quanto menor o BPC, melhor o modelo prevê o texto. Além da métrica, cada modelo gerou amostras a partir de um mesmo *prompt* ("Uma noite destas, vindo da cidade", início de *Dom Casmurro*), com temperatura 0,8, para análise qualitativa.

V. RESULTADOS

A. Curvas de treino

A Fig. 1 mostra as perdas de treino e validação. Nos três casos, a perda inicial corresponde ao esperado: 5,07 em E1 ($\ln 144 = 4{,}97$), 10,91 em E3 ($\ln 50\,304 = 10{,}83$) e 5,19 em E6, valor muito abaixo do acaso porque o GPT-2 já possui conhecimento de linguagem antes de ver o corpus. A Tabela IV consolida os resultados.

B. Comparação quantitativa

E1 atingiu 1,204 de perda de validação (1,737 BPC) ao fim das 5000 iterações, correspondendo a atribuir, em média geométrica, cerca de 30% de probabilidade ao caractere correto entre 144 possíveis. A diferença entre treino e validação foi pequena (0,08), sem sinal de sobreajuste, e o melhor ponto foi o último, indicando que o modelo ainda não havia convergido. O treino levou cerca de 10 minutos.

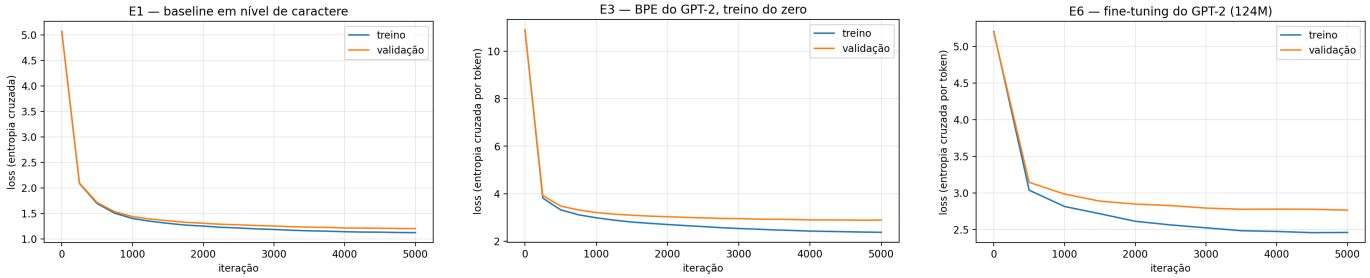


Figura 1. Perda (entropia cruzada por token) de treino e validação ao longo das 5000 iterações. Esquerda: E1 (caractere). Centro: E3 (BPE, do zero). Direita: E6 (*fine-tuning* do GPT-2). As escalas verticais diferem porque a perda é medida por token de vocabulários distintos.

E3 atingiu 1,653 BPC, 4,8% melhor que E1. Dois fenômenos se destacam. Primeiro, o sobreajuste apareceu: a lacuna entre treino e validação chegou a 0,50 e a perda de validação estabilizou ao final (2,8853 na iteração 4750, 2,8875 na 5000). Isso é coerente com o número de passagens pelo corpus: com BPE, o treino tem apenas 4,2M de tokens, e as 5000 iterações o percorreram cerca de 19 vezes, contra 7,6 em E1. Segundo, o custo por iteração quase triplicou, porque a camada de saída passou a decidir entre 50 mil tokens em vez de 144.

E6 obteve o melhor resultado, 1,586 BPC, 4,1% melhor que E3 e 8,7% melhor que E1. O GPT-2 sem ajuste apresentava 2,975 BPC, bem pior que qualquer modelo treinado do zero, o que quantifica o pouco português presente em seu pré-treinamento; ainda assim, após o ajuste, o conhecimento prévio se converteu em vantagem. A lacuna treino–validação (0,30) foi menor que a de E3, mesmo com um modelo quatro vezes maior e o mesmo número de passagens pelo corpus, sugerindo que os pesos pré-treinados atuam como regularizador. A perda de validação ainda caía na última avaliação (2,7789 na iteração 4500, 2,7671 na 5000), com uma oscilação na iteração 4000. O custo foi o maior: 1,15 s por iteração, cerca de 1h40 de treino.

C. Análise qualitativa

As amostras a seguir foram geradas com o mesmo *prompt* e temperatura, e são representativas do comportamento de cada modelo.

E1. “Uma noite destas, vindo da cidade, marchara-lhe diante das pernas do tio, que lhe não pareceu molesta. [...] e disse que Estela fosse a si mesma. / CAPÍTULO XIX / A DESENLADO / Punha a incumbência de algum tempo com o almoço [...]”. O modelo aprendeu a estrutura de romance (capítulos numerados com título em maiúsculas), a colocação pronominal de época (“lhe não pareceu”) e nomes do universo do autor (Estela, de *Iaiá Garcia*; Flora, de *Esaú e Jacó*; Rua do Ouvidor). Como prevê letra a letra, produz palavras inexistentes (“compatrimônio”, “trovoa”, “galons”) e erros de concordância (“A DESENLADO”).

E3. “Uma noite destas, vindo da cidade [...] A moça estremeceu. / — Tu és tu! não me aborrecerás; [...] / — Adivinha que é? / — Eu vivia assim cedo. / — Não creio; mas não sei o que for.” e “Era um sonho singular. Ainda assim, era verdade, a imagem da fortuna, a de um sonho de mistérios; mas, principalmente, era a flor da imaginação.” Quase não há palavras inventadas (apenas “vardinha”), os diálogos são mais longos e naturais e algumas frases têm ritmo machadiano, embora sem coerência de enredo.

E6. “Uma noite destas, vindo da cidade, Estêvão ia descobrir-lhe a mão. / — Desde que fui eu, disse ele consigo, Estêvão morreu? / — Não viria hoje em casa, disse o padre. / Estêvão fez um gesto de ameaça [...] / — Oh! não! disse ele ao padre com gesto fraco e grave. / — Ficou? perguntou Estêvão. / — Não; foi o resto. / Estêvão sorriu.” O diálogo mantém os mesmos personagens (Estêvão, de *A Mão e a Luva*, e um padre) por várias falas, com verbos dicendi e gestos intercalados à maneira do autor, algo que E1 e E3 não sustentam. Persistem frases sem sentido e, na execução mais curta de E6, repetições literais (“Parece que não conheço nada. Parece que não conheço nada.”).

D. Discussão

Os resultados respondem às três perguntas iniciais. (1) Um GPT de 10M de parâmetros em nível de caractere, treinado em 10 minutos, captura ortografia, pontuação, estrutura de diálogo e vocabulário do autor, mas não semântica de longo alcance. (2) A tokenização por subpalavras melhora o BPC e a qualidade lexical das amostras, ao custo de treino mais caro e maior sobreajuste, já que o mesmo orçamento em tokens cobre menos texto distinto. (3) O pré-treinamento do GPT-2 é benéfico mesmo vindo de outro idioma, produzindo o melhor BPC e os diálogos mais coerentes, embora exija um orçamento de treino comparável ao dos modelos do zero para superar seu ponto de partida desfavorável.

Duas ressalvas são necessárias. A comparação E1–E3 não isola perfeitamente o tokenizador: E3 tem três vezes mais parâmetros (pela tabela de *embeddings*) e um contexto efetivo 2,5 vezes maior em caracteres. A conclusão correta é que a configuração com BPE foi melhor, sem atribuir o ganho exclusivamente à tokenização. Analogamente, E6 difere de E3 também em profundidade e largura, fixadas pelos pesos do GPT-2. Além disso, todos os números reportados são do conjunto de *validação*, usado também para selecionar o melhor *checkpoint*; o conjunto de teste, reservado desde a divisão do corpus, não chegou a ser avaliado, de modo que os BPC podem ser levemente otimistas.

VI. Extensão: Perguntas no Universo Machado de Assis

Os modelos treinados são continuadores de texto: dado um início, produzem uma sequência plausível no estilo do autor. Eles não respondem a perguntas de forma confiável por três razões. Primeiro, não passaram por ajuste com instruções: diante de "Quem narra *Dom Casmurro*?", tendem a continuar a frase em vez de respondê-la. Segundo, um modelo de 10M a 124M de parâmetros tem capacidade limitada para memorizar fatos específicos das obras. Terceiro, mesmo quando produz uma resposta, não há garantia de que ela esteja ancorada no texto, e o modelo pode alucinar personagens ou enredos.

A extensão mais direta é a **geração aumentada por recuperação** (RAG) [9], que reutiliza o corpus já construído:

1) *Segmentação.* O corpus limpo é dividido em trechos por capítulo ou conto, cada um com metadados de obra, gênero e ano, o que a estrutura de um parágrafo por linha e os metadados existentes já permitem.
2) *Indexação.* Cada trecho é representado por um vetor de um modelo de *embeddings* multilíngue (por exemplo, bge-m3 ou multilingual-e5) e armazenado em um índice vetorial (FAISS).
3) *Recuperação.* Dada uma pergunta, os k trechos mais similares são recuperados.
4) *Geração.* Um modelo de linguagem com capacidade de seguir instruções recebe a pergunta e os trechos e produz uma resposta citando obra e capítulo, o que permite verificação pelo usuário.

Nesse arranjo, o modelo ajustado neste trabalho pode ser usado como gerador de paráfrases estilísticas ou de continuações "à maneira de Machado", enquanto a resposta factual é delegada a um modelo maior ancorado no texto recuperado. A avaliação pode ser feita com um conjunto de perguntas elaboradas manualmente, como "O que é o Humanitismo?" ou "Quem são os gêmeos de *Esaú e Jacó*?", medindo acurácia e presença da evidência correta entre os trechos recuperados.

Uma segunda via, complementar, é o **ajuste supervisionado com instruções** (SFT) [10]: construir pares pergunta–resposta sobre as obras e continuar o treino do modelo E6 sobre eles. Isso ensinaria o formato de pergunta e resposta, mas não resolveria a limitação de capacidade nem a ausência de ancoragem; por isso, o SFT é mais promissor aplicado a um modelo maior pré-treinado em português e combinado ao RAG. Outras extensões incluem geração condicionada por gênero literário (prefixando cada obra com um token de gênero no treino) e a comparação com versões maiores do GPT-2 ou com modelos pré-treinados em português, inviáveis na GPU disponível.

VII. Limitações e Trabalhos Futuros

Além das ressalvas da Seção V-D, destacam-se: (i) a avaliação final no conjunto de teste, com BPC calculado sobre o texto completo em vez de estimado por amostragem de *batches*; (ii) a inclusão de um tokenizador BPE treinado no próprio corpus, que reduziria a fragmentação do português e isolaria melhor o efeito do vocabulário; (iii) métricas de memorização (fração de n-gramas longos do texto gerado presentes no treino) e de diversidade (*distinct-*n), além de avaliação de estilo por juiz automatizado ou por teste cego com leitores; (iv) a extensão do treino de E1 e E6, cujas perdas de validação ainda caíam ao final; e (v) o tratamento das quebras de estrofe na poesia, perdidas pela representação de um parágrafo por linha.

VIII. Conclusão

Este trabalho construiu um corpus limpo e documentado com 243 obras originais de Machado de Assis (13,4M de caracteres) e o utilizou para treinar e comparar três modelos de linguagem estilo GPT-2 com o nanoGPT. Sob o mesmo orçamento de treino, a tokenização por subpalavras reduziu o BPC de 1,737 para 1,653, e o *fine-tuning* do GPT-2 pré-treinado o reduziu a 1,586, apesar de o modelo original apresentar 2,975 BPC no português do autor. As amostras confirmam a progressão: do domínio da ortografia e da estrutura de diálogo em E1 à manutenção de personagens ao longo de várias falas em E6. O sistema pode ser estendido para responder a perguntas sobre o universo machadiano combinando o corpus já estruturado a um pipeline de recuperação e a um modelo capaz de seguir instruções.

Atribuição de Código

O código foi entregue em um *notebook* do Google Colab, disponível em https://github.com/RobertorNeto/MachadoGPT. São de terceiros: a arquitetura do modelo, o laço de treino, o carregamento dos pesos do GPT-2 e o script de amostragem, do nanoGPT (A. Karpathy, licença MIT, commit `3adf61e`); os pesos do GPT-2 (OpenAI, via Hugging Face); e o tokenizador BPE (`tiktoken`, OpenAI). A tokenização em nível de caractere foi adaptada do exemplo `shakespeare_char` do nanoGPT, e a codificação BPE, do `prepare.py` de `openwebtext`. São de autoria própria: o pipeline de download, limpeza e estatísticas do corpus; o algoritmo de divisão por obra; a construção do vocabulário de caracteres a partir do treino; os arquivos de configuração dos experimentos; a extração das curvas de treino e a comparação em bits por caractere.

Referências

[1] A. Vaswani *et al.*, "Attention is all you need," em *Advances in Neural Information Processing Systems*, vol. 30, 2017. arXiv:1706.03762.

[2] T. B. Brown *et al.*, "Language models are few-shot learners," em *Advances in Neural Information Processing Systems*, vol. 33, 2020. arXiv:2005.14165.

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, 2019.

[4] A. Karpathy, "nanoGPT," 2023. Disponível em: https://github.com/karpathy/nanoGPT. Tutorial: "Let's build GPT: from scratch, in code, spelled out," YouTube, 2023.

[5] R. Sennrich, B. Haddow e A. Birch, "Neural machine translation of rare words with subword units," em *Proc. ACL*, 2016.

[6] S. Bird, E. Klein e E. Loper, *Natural Language Processing with Python*. O'Reilly, 2009. Pacote `machado` do NLTK Data: https://github.com/nltk/nltk_data.

[7] J. M. Machado de Assis, *Obra Completa*, 3 vols. Rio de Janeiro: Nova Aguilar, 1994. Versão eletrônica: Ministério da Educação, http://machado.mec.gov.br.

[8] I. Loshchilov e F. Hutter, “Decoupled weight decay regularization,” em *Proc. ICLR*, 2019.
[9] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” em *Advances in Neural Information Processing Systems*, vol. 33, 2020.
[10] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” em *Advances in Neural Information Processing Systems*, vol. 35, 2022.